

Lists - an Introduction

Section 4

Chapter 2

An Introduction to Programming Using Python
David Schneider

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Lists

- Topics
 - List objects
 - Nested lists
 - Indices and slices
 - List functions and methods

The `list` Object

- A list is an ordered sequence of Python objects
 - Objects can be of any type (including other lists).
 - Lists within another list are called **nested** lists.
 - Objects do not have to all be the same type.
 - Constructed by writing items enclosed in square brackets ... items separated by commas.

```
team = ["Seahawks", 2014, "CenturyLink Field"]  
nums = [5, 10, 4, 5]  
words = ["spam", "ni"]
```

List Index

- Use [] to index items
 - L1 = [3, 'hi', 4.5, [1,2,3]] # L1 contains a **nested** list
 - >>> L1[1]
 - 'hi'
 - >>> L1[3][0]
 - 1
 - >>> L1[6]
 - Error
 - >>> L1[-1]
 - [1,2,3]

Slices

- A *slice* of a list is a sublist specified with colon notation
 - Analogous to a slice of a string
- Table 2.6 Meanings of slice notations

Slice Notation	Meaning
<code>list1[m:n]</code>	list consisting of the items of <i>list1</i> having indices <i>m</i> through <i>n</i> – 1
<code>list1[:]</code>	a new list containing the same items as <i>list1</i>
<code>list1[m:]</code>	list consisting of the items of <i>list1</i> from <code>list1[m]</code> through the end of <i>list1</i>
<code>list1[:m]</code>	list consisting of the items of <i>list1</i> from the beginning of <i>list1</i> to the element having index <i>m</i> – 1

Slices

- Table 2.7 Examples of slices where `list1 = ['a', 'b', 'c', 'd', 'e', 'f']`.

Example	Value
<code>list1[1:3]</code>	<code>['b', 'c']</code>
<code>list1[-4:-2]</code>	<code>['c', 'd']</code>
<code>list1[:4]</code>	<code>['a', 'b', 'c', 'd']</code>
<code>list1[4:]</code>	<code>['e', 'f']</code>
<code>list1[:]</code>	<code>['a', 'b', 'c', 'd', 'e', 'f']</code>
<code>del list1[1:3]</code>	<code>['a', 'd', 'e', 'f']</code>
<code>list1[2:len(list1)]</code>	<code>['c', 'd', 'e', 'f']</code>
<code>(list1[1:3])[1]</code>	<code>'c'</code> (This expression is usually written as <code>list1[1:3][1]</code>)
<code>list1[3:2]</code>	<code>[]</code> , the list having no items; that is, the empty list

Indexing, Deleting, and Slicing Out of Bounds

- Python does *not* allow out of bounds indexing for individual items in lists and tuples
 - But *does* allow it for slices
- Given `list1 = [1, 2, 3, 4, 5]`
 - Then

```
print(list1[7])  
print(list1[-7])  
del list1[7]
```


Indexing, Deleting, and Slicing Out of Bounds

- If left index in slice too far negative
 - Slice will start at the beginning of the list
- If right index is too large,
 - Slice will go to the end of the list.

```
list1[-10:10] is [1, 2, 3, 4, 5]  
list1[-10:3] is [1, 2, 3]  
list1[3:10] is [4, 5]  
del list1[3:7] is [1, 2, 3]
```


Your Turn ...

- `>>> cities = [['windsor', 'toronto'], ['calgary', 'edmonton'], ['montreal'], ['victoria', 'vancouver']]`
- `>>> cities[1:3]` *# slice notation*
 - `[['calgary', 'edmonton'], ['montreal']]`
- `>>> cities[1][-2]`
 - `'calgary'`
- `>>> cities[0][1][-3]`
 - `'n'`
- `>>> len(cities[2])`
 - `1`
- `>>> cities[0][0]`
 - `'windsor'`
- `>>> cities`
 - `[['windsor', 'toronto'], ['calgary', 'edmonton'], ['montreal'], ['victoria', 'vancouver']]`

Immutable and Mutable Objects

- Objects changed in place are *mutable*
- Objects that *cannot* be changed in place are *immutable*
 - List objects are ***mutable*** objects. Numbers and strings are ***immutable*** objects.

Immutable and Mutable Objects

```
L1 = [1, 2, ['abc', 3.5], 7.56, 4]
S1 = "Welcome to COM 2067"
L1[1] = 99
L1[2][1] = 'XYZ'
print('Printing updated list L1:', L1)

S1[13] = 'P'
print('Printing updated string S1:', S1)
```

<https://IndigoFrizzyDowngrade.ajael.repl.run>



```
Python 3.6.1 (default, Dec 2015, 13:05:11)
[GCC 4.8.2] on linux
Printing updated list L1: [1, 99, ['abc', 'XYZ'], 7.56, 4]
Traceback (most recent call last):
  File "main.py", line 7, in <module>
    S1[13] = 'P'
TypeError: 'str' object does not support item
assignment
```



The `list` Object

- Table 2.5 List operations

```
team = ["Seahawks", 2014, "CenturyLink Field"]
nums = [5, 10, 4, 5]
words = ["spam", "ni"]
```

Function or Method	Example	Value	Description
<code>len</code>	<code>len(words)</code>	2	number of items in list
<code>max</code>	<code>max(numbers)</code>	10	greatest (items must have same type)
<code>min</code>	<code>min(numbers)</code>	4	least (items must have same type)
<code>sum</code>	<code>sum(nums)</code>	42	total (items must be numbers)
<code>count</code>	<code>nums.count(5)</code>	2	number of occurrences of an object
<code>index</code>	<code>nums.index(4)</code>	2	index of first occurrence of an object
<code>reverse</code>	<code>words.reverse()</code>	<code>["ni", "spam"]</code>	reverses the order of the items

The `list` Object

- Table 2.5 List operations

```
team = ["Seahawks", 2014, "CenturyLink Field"]
nums = [5, 10, 4, 5]
words = ["spam", "ni"]
```

reverse	<code>words.reverse()</code>	<code>["ni", "spam"]</code>	reverses the order of the items
clear	<code>team.clear()</code>	<code>[]</code>	<code>[]</code> is the empty list
append	<code>nums.append(7)</code>	<code>[5, 10, 4, 5, 7]</code>	inserts object at end of list
extend	<code>nums.extend([1, 2])</code>	<code>[5, 10, 4, 5, 1, 2]</code>	inserts new list's items at end of list
del	<code>del team[-1]</code>	<code>["Seahawks", 2014]</code>	removes item with stated index
remove	<code>nums.remove(5)</code>	<code>[10, 4, 5]</code>	removes first occurrence of an object
insert	<code>nums.insert(1, "wink")</code>	<code>["spam", "wink", "ni"]</code>	insert new item before item of given index
+	<code>['a', 1] + [2, 'b']</code>	<code>['a', 1, 2, 'b']</code>	concatenation; same as <code>['a', 1].extend([2, 'b'])</code>
*	<code>[0] * 3</code>	<code>[0, 0, 0]</code>	list repetition

List Operations

- `L = [1,2,3,[4,5,'a','b'], 'hello', '6', 7]`
 - `L.append([8,9])` →
 - `[[1, 2, 3, [4, 5, 'a', 'b'], 'hello', '6', 7, [8, 9]]`
 - `L + [8,9]` →
 - `[1, 2, 3, [4, 5, 'a', 'b'], 'hello', '6', 7, 8, 9]`
 - `L.index(3)` →
 - `2`
 - `len(L)` →
 - `7`

NOTE: This is not an exhaustive list. We are always starting with **initial** value of L.

List Operations

- `L = [1,2,3,[4,5,'a','b'], 'hello', '6', 7]`
 - `L.insert(2, [8,9]) →`
 - `[1, 2, [8, 9], 3, [4, 5, 'a', 'b'], 'hello', '6', 7]`
 - `L.pop(4) →`
 - `'hello' # Updated L → ([1,2,3,[4,5,'a','b'], '6', 7])`
 - `L.reverse() →`
 - `[7, '6', 'hello', [4, 5, 'a', 'b'], 3, 2, 1]`

NOTE: This is not an exhaustive list. We are always starting with **initial** value of L.

End

Section 4

Chapter 2

An Introduction to Programming Using Python
David Schneider

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.